

**Az SQL adatbázisnyelv: Az adatdefiníciós nyelv (DDL) és az adatmanipulációs nyelv (DML).  
Relációsémák definiálása, megszorítások típusai és létrehozásuk. Adatmanipulációs  
lehetőségek és lekérdezések.**

## **SQL ADATBÁZISNYELV: AZ ADATDEFINÍCIÓS NYELV (DDL) ÉS AZ ADATMANIPULÁCIÓS NYELV (DML)**

### **SQL (Structured Query Language) adatbázis nyelv**

- 1974, Raymond S. Boyce és Donald D. Chamberli
  - első neve az SEQUEL (Structured English Query Language)
- relációs adatbázis-kezelés szabványos leíró és lekérdező nyelve
- lehetővé teszi az adatok strukturált létrehozását, módosítását, lekérdezését és törlését
- az SQL egy **deklaratív nyelv**
  - nem algoritmikus nyelv, tehát nincsenek algoritmikus adatszerkezetek, se vezérlési műveletek (for, while)
  - változókat sem használ az adatok tárolására az utasításokon belül, a kimenet pedig mindig egy eredmény tábla
  - **deklaratív szemlélet:** a felhasználó azt fogalmazza meg, hogy mit szeretne eredményül kapni (például „mely dolgozók keresnek többet az átlagnál”), de nem kell leírnia a végrehajtás algoritmikus lépéseit
    - hogy ez fizikailag hogyan valósul meg, az adatbázis-kezelő (DBMS) optimalizálója dönti el, a programozónak ennek a működését nem kell ismernie
- a nyelv utasításait funkcionálisan több csoportra bontjuk:
  - **adatdefiníciós nyelv (DDL - Data Definition Language)**
    - adatstruktúrát (táblák, indexek, nézetek) definiáló utasítások
    - utasítások: CREATE, ALTER, DROP
  - **adatmanipulációs nyelv (DML - Data Manipulation Language)**
    - adatok aktualizálása, manipulálása (beszúrás, módosítás, törlés)
    - utasítások: INSERT, UPDATE, DELETE
  - **adatlekérdező nyelv (DQL - Data Query Language)**
    - lekérdezések
    - utasítás: SELECT
  - **adatkezelő nyelv (DCL - Data Control Language)**
    - vezérlés (jogosultságkezelés, tranzakció kezelés)
    - jogosultságkezelő utasítások: GRANT, REVOKE
    - tranzakciókezelő utasítások: COMMIT, ROLLBACK
- SQL egy szabvány, azaz alapok egyeznek, de szoftverenként (MySQL, Oracle, IBM, PostgreSQL) eltérő dolgok is lehetnek
  - folyamatos fejlődés, kiegészítő nyelvek, oop, stb
- SQL egy **szigorúan típusos nyelv**, minden oszlopnak típust kell adnunk:
  - karakteres adattípusok
    - CHAR(n):  $n$  hosszúságú karaktersorozat
    - VARCHAR(n): max  $n$  hosszúságú karaktersorozat

- TEXT: hosszabb szövegek tárolására
- numerikus adattípusok
  - INTEGER / INT: egész szám
  - REAL, FLOAT, DOUBLE: valós (lebegőpontos) szám
  - DECIMAL(n, [d]): *n* jegyű decimális szám, *d* tizedesjeggyel
    - *d* megadása opcionális
- dátum adattípusok
  - DATE: dátum (év, hónap, nap)
  - TIME: idő (óra, perc, másodperc)
  - TIMESTAMP: dátum + idő
- egyéb
  - BOOLEAN: logikai értékek (TRUE, FALSE)
  - BLOB: bináris adatok (pl: kép, fájl)
  - NULL: hiányzó, definiálatlan érték jelölése
- szintaxis
  - kis és nagybetű egyenértékű az SQL kulcsszavak esetén (case-insensitive)
    - (de pl: tábla neveknél és oszlopneveknél nem mindegyiknél van így)
  - utasításokat ; -vel kell lezárni
  - nincsenek változók, csak táblák és azon belül oszlopnevek
  - *tábla.oszlop\_nev* vagy csak *oszlop\_nev*, ha egyértelmű
  - táblák és oszlopok alias nevét AS-el tudjuk meghatározni
    - *tábla* AS *t1*
    - *oszlop\_neve* AS *o1*
  - string összefűzés: + vagy ||
  - szöveg konstansok '' között (szimpla aposztrófjelek)
  - = , < , <= , > , >= , != (<>)
  - logika
    - AND, OR, NOT
    - TRUE, FALSE, NULL (unknown)
      - 3 rendű, **háromértékű logika**
    - *x* IS 15 (*x* = 15)
    - *x* BETWEEN 15 AND 20
    - LIKE "%minta\_ "
      - % : tetszőleges karaktersorozat
      - \_ (aláhuzás): tetszőleges karakter
    - *x* IN halmaz
      - benne van-e *x* az adott halmazban
      - ("A", "DSF") vagy egy lekérdezés a halmaz
    - *x* != ALL(81, 83, 100)
      - akkor ad igaz értéket, ha a halmaz minden elemére igaz érték ad vissza, tehát itt *x* != 81 -el és a többivel se
      - != helyett bármilyen relációsjel mehet

- $x = \text{ANY}(15, 20)$ 
  - akkor ad igaz értéket, ha a halmaz legalább egyik elemére igaz értéket adna, tehát ha  $x = 20$ , akkor igazat dob
  - itt is cserélhető a relációs jel
- EXISTS halmaz
  - igazat ad, ha a halmaz nem üres, pl lekérdezésünkkel kaptunk-e vissza adatot

## Az adatdefiníciós nyelv (DDL)

- DDL utasítások segítségével határozzuk meg az adatbázis szerkezetét, a sémákat és az objektumokat
  - az utasítások nem az adatsorokon, hanem a táblák metaadatain dolgoznak
- **Adatbázis létrehozása és használata**
  - CREATE DATABASE adatbázis\_neve;
  - SHOW DATABASES;
    - listázza az elérhető adatbázisokat
  - USE adatbázis\_neve;
    - aktuális adatbázis ez lesz
  - DROP DATABASE adatbázis\_neve;
- **Relációsémák létrehozása (CREATE)**
  - CREATE TABLE tabla\_neve (
 

oszlop\_neve1 típus [oszlop\_feltétel],

oszlop\_neve2 típus [oszlop\_feltétel],

[tablafeltétel1, tablafeltétel2]

);
  - létrehoz egy új táblát és definiálja annak oszlopait és azok típusait
    - SQL szigorúan típusos ezért kell minden oszlopnak típust adni
- **Megszorítások (Constraints)**
  - két típusa van **oszlopfeltétel** (rögtön a típus után) és **tablafeltétel** (a felsorolás végén)
    - tablafeltételeknél ha egyedi nevet akarunk a megszorításnak akkor így adjuk meg: **CONSTRAINT nev [feltétel]**
  - csak oszlopfeltételek, megszorítások
    - **AUTO\_INCREMENT**
      - automatikus növeléses sor, csak kulcs lehet (pl: id)
    - **DEFAULT érték\_valtozo**
      - kezdő érték megadása
    - **NOT NULL**
      - kötelező értékadás, az attribútum, oszlop értéke nem lehet NULL
    - **REFERENCES tabla\_neve(oszlop\_neve)**
      - azaz külső kulcs

- legfontosabb megszorítások:

- **PRIMARY KEY (elsődleges kulcs)**

- sorok egyértelműen azonosíthatóak ezen attribútum, oszlop értékével
  - nem lehet NULL és egy táblának egy PRIMARY KEY-e lehet
- oszlop feltétel: `id INT PRIMARY KEY`
- táblafeltétel: `PRIMARY KEY (oszlop_lista)`
  - `PRIMARY KEY (id, személyi_szam)` lista adható meg, akár csak 1 elemmel
  - ha összetett az elsődleges kulcs akkor csak táblafeltételként lehet megadni

- **UNIQUE**

- egyediséget jelöl, de értéke értéke akár NULL is lehet és több oszlop is megkaphatja (nem mint a PRIMARY KEY-t)
- oszlop feltétel: `oszlop UNIQUE`
- táblafeltétel: `UNIQUE (oszlop_lista)`

- **CHECK(feltétel)**

- a mező, attribútum értékének eleget kell tennie a feltételnek
- oszlop feltétel: `age INT CHECK (age >= 18);`
- táblafeltétel: `CHECK (start_date < end_date)`

- **FOREIGN KEY**

- hivatkozási integritás
- egy másik tábla elsődleges kulcsára hivatkozik, biztosítja a táblák közötti kapcsolat érvényességét
- CSAK táblafeltétel és akkor használjuk általában, ha a kulcs összetett, azaz több attribútumból áll
- `FOREIGN KEY (oszlop_lista)`  
`REFERENCES tabla_neve(oszlop_lista) [ON:feltételek]`
- **referenciális akciók** (ON feltételek)
  - megadhatjuk, hogy mi történjen, ha hivatkozott (szülő) rekordot törlik vagy módosítják
  - **ON DELETE kulcsszo** és **ON UPDATE kulcsszo**

- **RESTRICT**

- nem módosítható vagy törölhető a hivatkozott külső kulcs, ha van egy táblába rámutató külső kulcs

- **SET NULL**

- ha módosítjuk vagy töröljük a hivatkozott külső kulcs értékét, akkor azok amik arra hivatkoznak, rámutatnak az értékük NULL-ra módosul

## ■ CASCADE

- ha módosítjuk a hivatkozott külső kulcs értékét, akkor automatikusan módosítjuk erre az értékre a rámutató mezőket is
- ha töröljük a hivatkozott külső kulcs értékét, akkor a rámutató mezőket is töröljük

## • Sémák módosítása és törlése

- a már létező sémákat az **ALTER TABLE** utasítással módosíthatjuk
  - **ALTER TABLE tabla\_neve ADD (oszlop\_nev típus [feltétel]);**
    - új oszlop hozzáadása a sémához
    - pl: ALTER TABLE Customers ADD email VARCHAR(255);  
ALTER TABLE t1 ADD CONSTRAINTS nev [tablafeltétel];
  - **ALTER TABLE tabla\_neve MODIFY (oszlop\_nev típus [feltétel]);**
    - meglévő oszlop, attribútum módosítása
    - pl: ALTER TABLE Customers MODIFY email varchar(32);
  - **ALTER TABLE tabla\_neve DROP COLUMN (oszlop\_lista);**
    - oszlop(ok), attribútum(ok) törlése
    - pl: ALTER TABLE Customers DROP COLUMN (email, age);
- teljes tábla törlésére a **DROP TABLE** utasítás van
  - **DROP TABLE tabla\_neve;**

## Az adatmanipulációs nyelv (DML)

- a DML utasítások segítségével kezeljük a táblákban tárolt konkrét adatokat
- **Beszúrás (INSERT INTO)**
  - új sorok felvitelére szolgál
  - **INSERT INTO tabla\_neve (oszlop\_lista) VALUES (ertekek);**
    - amennyiben a tábla minden oszlopába szúrunk be értéket akkor elhagyható az oszlop lista
      - pl: INSERT INTO User VALUES ("nev@gmail.com", "nev")
  - ha az INSERT megsért egy UNIQUE megszorítást akkor a DBMS hibaüzenetet ad és a beszúrás nem történik meg
- **Rekord módosítása (UPDATE)**
  - meglévő sor, rekord módosítására szolgál
  - **UPDATE tabla\_neve SET oszlop1 = ertekek, o2 = ertekek2 [WHERE feltétel];**
    - fontos, ha nincs feltétel akkor az összes sorba módosítja az adott oszlopok értékét
- **Rekord törlése (DELETE FROM)**
  - meglévő rekord, sor törlésére szolgál
  - **DELETE FROM tabla\_neve [WHERE feltétel];**
    - fontos, ha nincs feltétel a tábla összes sora törlésre kerül

## Lekérdezések (DQL)

- SQL leggyakrabban használt része a **SELECT** utasítás
  - adatok kinyerésére szolgál
  - **SELECT** [**DISTINCT**] oszlopok  
FROM táblák  
[**WHERE** feltétel]  
[**GROUP BY** oszlopok]  
[**HAVING** feltétel]  
[**ORDER BY** oszloplista]
  - szintaktikai sorrend mellett van még egy logikai sorrend is:
    1. **FROM - Descartes szorzat**
      - a forrástáblák kiválasztása és összekapcsolása (**JOIN**)
    2. **WHERE - Szelekció**
      - soronkénti, rekordonkénti szűrés
      - itt aggregált értékek (**AVG**, **MAX**, **MIN**, **COUNT**, **SUM**) nem használhatók mert még nem történt meg a **GROUP BY**, az alias nevek sem használhatók mert az a **SELECT**-ben történnek a projekció során és az még később lesz (5.)
    3. **GROUP BY - Csoportonkénti összevonás**
      - a szelekció után megmaradt sorok csoporotosítása kijelölt oszloppokként
        - egy csoportba azok a sorok tartoznak, melyeknél oszloplista értéke azonos
    4. **HAVING - Csoport szelekció**
      - csoportok szűrése, itt már aggregált értékek használhatók
      - pl: **HAVING AVG(price) > 50000**
    5. **SELECT - Projekció**
      - a megjelenítendő oszlopok kiválasztása
      - opcionálisan **DISTINCT** kulcsszó is megjelenhet, ilyenkor a duplikátumok kerülnek kiszűrésre
    6. **ORDER BY - Sorbarendeázés**
      - a végeredmény rendezése
      - **ASC** - növekvő sorrend, **DESC** - csökkenő sorrend
- **Táblák összekapcsolása (JOIN)**
  - **természetes összekapcsolás**
    - csak azokat a sorokat adja vissza, amelyeket egy attribútum alapján össze tud kapcsolni, azaz mindkét táblában egyezés van
    - metszet
    - **FROM t1 INNER JOIN t2 ON t1.id = t2.code**
    - **FROM t1, t2 WHERE t1.id = t2.code**
    - **FROM t1 NATURAL JOIN t2**
      - feltételezi, hogy a kulcs és külső kulcs neve megegyezik

- **baloldali külső kapcsolás**
  - azokat a sorokat adja vissza, amelyeket egy attribútum alapján össze tud kapcsolni, illetve a kapcsolás bal oldalán lévő tábla egyéb összes sorát
    - amelyik sornak nem lesz párja, ott a másik oldalból származó adatoknál NULL érték lesz
  - FROM t1 LEFT (OUTER) JOIN t2 ON t1.id = t2.code
- **jobboldali külső kapcsolás**
  - azokat a sorokat adja vissza, amelyeket egy attribútum alapján össze tud kapcsolni, illetve a kapcsolás jobb oldalán lévő tábla egyéb összes sorát
    - amelyik sornak nem lesz párja, ott a másik oldalból származó adatoknál NULL érték lesz
  - FROM t1 RIGHT (OUTER) JOIN t2 ON t1.id = t2.code
- **teljes külső kapcsolás**
  - mindkét oldalról veszi a sorokat és ahol lehet ott összekapcsolja őket
    - amelyik sornak nem lesz párja, ott a másik oldalból származó adatoknál NULL érték lesz
  - FROM t1 FULL OUTER JOIN t2 ON t1.id = t2.code
- **theta összekapcsolás**
  - nem kell legyen külső kulcs kapcsolat a két tábla között
  - FROM t1, t2 WHERE t1.ferohely < t2.kapacitas
- **SELF JOIN**
  - egy tábla összekapcsolása önmagával (pl: Dolgozó és Főnöke, a főnök is egy dolgozó)
  - pl: FROM Dolgozo AS d1, Dolgozo AS d2
- **Aggregáló függvények (összesítő függvények)**
  - statisztikák készítésére jók
  - AVG, SUM, MIN, MAX, COUNT
- **Halmaz műveletek**
  - unió
    - (alkérdés) UNION (alkérdés)
  - metszet
    - (alkérdés) INTERSECT (alkérdés)
  - különbség
    - (alkérdés) EXCEPT (alkérdés)
    - néhol EXCEPT helyett MINUS kulcsszó van

- **Allekérdezések**

- egy lekérdezésen belül egy másik SELECT
- típusai
  - **nem korrelált**
    - önmagában is lefuttatható és csupán egyszer fut le
    - pl: WHERE fizetes > (SELECT AVG(fizetes) FROM Berek)
  - **korrelált**
    - az allekérdezés hivatkozik a külső lekérdezés egy oszlopára
    - lassabb, mint a nem korrelált
      - minden külső sorra újra lefut

- **Nézettáblák (virtuális tábla, view)**

- nem tárolnak adatokat
- egy transzormációs formula, ami segítségével a tárolt táblák egy meghatározott lekérdezés adott eredményét kapjuk meg
- CREATE VIEW tábla\_neve [(oszlop\_lista)] AS alkérdés
  - pl: CREATE VIEW view\_name AS  
SELECT column1, column2  
FROM table\_name  
WHERE condition;
- alkalmazási lehetőségek
  - bizonyos adatok elrejtése egyes felhasználók elől (adatbiztonság vagy egyszerűsítés céljából)
  - származtatott adattáblák létrehozása, melyek a törzsadatok módosításakor automatikusan módosulnak (pl: összegzőtáblák)